GOLLIS UNIVERSITY

FACULTY OF computer Studies

ICT DEPARTMENT

FINAL YEAR PROJECT

HOTEL BOOKING SYTEM

Abdisalaan Faysal Ali GU-ICT-C615
Kulmiye Mohamud Abdilaahi GU-ICT-C657
Adnan Abdirashid Mohamed GU-ICT-C572
Ayub Said Dahir GU-ICT-C701
Ahmed Abdiwasac Salah GU-ICT-C712
Mohamed Mohamud Adan GU-ICT-C624

Eng. Qalin Maal

A Final Year Project

Submitted to Faculty of Computing and Informatics, GOLLIS Institute of Technology, GOLLIS University, in Partial fulfillment of the requirement of the Degree of Bachelor of Science in

Computer Science

GOLLIS, Bosaso
November 2025/26

Table of Contents

LIST OF FIGURES

LIST OF TABLES

CHAPTER ONE
INTRODUCTION

1.1 Background of the Study

The rapid advancement of information technology, particularly mobile applications and internet-based services, has significantly transformed how people access services such as travel and accommodation. Hotel booking systems have become essential tools that enable users to search, compare, and reserve hotel rooms efficiently and conveniently.

In many developing regions, including Somalia, hotel booking is still largely conducted through traditional methods such as phone calls, physical visits, and manual record-keeping. These methods are inefficient, time-consuming, and prone to errors such as double booking and inaccurate data.

Therefore, there is a need for a modern, automated, and reliable solution that improves efficiency and enhances the user experience. This study aims to develop a Hotel Booking System using PHP and MySQL to provide a fast, secure, and easy-to-use system.

1.2 Statement of the Problem

Despite the availability of many hotels, customers still face challenges when booking rooms. These challenges include a lack of accurate and up-to-date information, time-consuming booking procedures, and the risk of booking errors such as double reservations.

Additionally, there is no centralized application that allows users to easily search, compare, and book hotels in one place. In regions like Somalia, the absence of efficient digital platform further complicates the booking process and reduces service quality.

Therefore, there is a need to develop a reliable and user-friendly hotel booking system that addresses these issues and improves overall service delivery.

1.3 Objectives of the Study

1.3.1 General Objective

The main objective of this project is to design and develop a hotel booking system that simplifies and improves the process of searching and booking hotel rooms.

1.3.2 Specific Objectives

- To develop a user-friendly platform interface
- To provide hotel search and filtering features
- To implement a secure booking system
- To store and manage user and booking data
- To include a review and rating application
- To provide booking confirmation notifications

1.4 Feasibility Study

1.4.1 Technical Feasibility

The project is technically feasible because the required technologies such as PHP and MySQL are readily available and widely used. The developers also have the necessary skills to implement the system successfully.

1.4.2 Economic Feasibility

The cost of developing the system is relatively low since open-source tools are used. Additionally, the system has the potential to generate economic benefits by increasing hotel bookings and customer satisfaction.

1.4.3 Operational Feasibility

The system is easy to use and accessible, making it suitable to a wide range of users. It can be operated on mobile devices, allowing users to book hotels anytime and anywhere.

1.4.4 Schedule Feasibility

The project can be completed within the allocated academic timeframe. Proper planning and scheduling will ensure timely completion of all development phases.

1.5 Scope of the Study

The system will focus on providing core functionalities such as user registration and login, hotel search and filtering, viewing hotel details including price, location, and reviews, and booking hotel rooms. It will also allow users to manage their profiles and provide feedback through a review and rating system.

However, the system will not include advanced features such as flight booking or full online payment integration due to time and resource limitations.

1.6 Significance of the Study

This project is significant because it provides users with a convenient way to book hotel rooms quickly and efficiently. It also helps hotel owners reach more customers and improve their services. Furthermore, it offers developers practical experience in application design and development.

1.7 Target Beneficiaries

The primary beneficiaries of this project include travelers, hotel owners, tourism companies, and students interested in system development and research.

1.8 Methodology

1.8.1 Data Collection Methods

Data for this project will be collected through interviews, observation, and document review.

1.8.2 System Development Process

The development process will include problem identification, requirement analysis, system design, development, testing, and implementation.

1.8.3 Development Model

The Agile (Iterative) model will be used to allow continuous improvement and flexibility during system development.

1.9 Limitations of the Study

The system depends on internet connectivity, which may limit access in some areas. Furthermore, payment integration may not be fully implemented, and real-time updates may occasionally experience delays.

CHAPTER TWO
REQUIREMENT ANALYSIS AND SPECIFICATION

2.1 Introduction

This chapter presents the system analysis and requirements needed to develop the Hotel Booking Management System. It describes the existing system, identifies its limitations, and presents the proposed system as a solution. In addition, it outlines both functional and non-functional requirements of the new system.

2.2 Existing System

Currently, many hotels rely on manual or partially computerized systems to manage bookings and customer information. Customers typically reserve rooms through phone calls, physical visits, or paper-based records.

These traditional methods are inefficient and often lead to delays, data loss, and inaccurate information. Since there is no centralized system, managing bookings and tracking room availability becomes difficult, especially during peak seasons.

2.3 Participants in the Existing System

The current system involves the following participants:

Customer: A person who wants to book a room

Staff: Responsible for recording bookings

Hotel Manager: Oversees the overall operations

These users rely on a manual system, which results in inefficiencies and errors.

2.4 Major Functions of the Existing System

The existing system performs basic operations such as recording customer bookings, manually assigning rooms, handling payments, and maintaining records using paper-based methods. These

functions are carried out without automation, which reduces efficiency and increases the risk of errors.

2.5 Problems of the Existing System

The existing system faces several major challenges. It is time-consuming and prone to human error due to manual data handling. There is no reliable database to store and manage information, making it difficult to retrieve records when needed.

Additionally, the system does not provide real-time updates, and customers are unable to easily check room availability. The absence of an online platform limits accessibility and reduces customer satisfaction. These issues lead to poor service delivery and overall customer dissatisfaction.

2.6 Proposed System

To address these challenges, a Hotel Booking Management System is proposed. This system provides an automated and user-friendly platform for managing hotel operations.

The proposed system will allow users to search for available rooms, book rooms, and process payments efficiently. It will also enable administrators to manage rooms, staff, and reports using a centralized database.

By using technologies such as PHP and MySQL, the system will ensure data accuracy, improve performance, and provide real-time access to information.

2.7 Requirements of the Proposed System

2.7.1 Functional Requirements

The Hotel Booking Management System shall provide the following functionalities:

- Manage rooms
- Manage room types and pricing
- Manage bookings
- Process and manage payments
- Search available rooms
- Filter rooms based on user preferences
- Manage staff information
- Generate system reports
- Display dashboard information
- Authenticate users, including login, registration, and password management

2.7.2 Non-Functional Requirements

The system should meet the following quality requirements:

Security: Ensure authentication and authorization
Performance: Provide fast response time
Usability: Ensure user-friendly interface
Reliability: Ensure system stability
Scalability: Support system growth

2.8 System Analysis

This section presents the modeling of both the existing and proposed systems using appropriate analysis techniques. These models help improve understanding of system requirements and how different components interact. Each model is directly related to the System Requirement Specification (SRS) and provides a clear representation of system functionality and behavior.

2.8.1 System Requirement Specification (SRS)

The System Requirement Specification defines the functional and non-functional requirements of the Hotel Booking Management System. These requirements are modeled using different diagrams to provide a clear understanding of the system processes and structure.

2.8.1.1 Use Case Model Diagram

The use case diagram represents the interaction between users (actors) and the system. It identifies the main functionalities that each user can perform.

Narrative Description

The main actors in the system are Customer, Admin, and Staff. The Customer can register, log in, search for rooms, make bookings, and perform payments. The admin manages rooms, staff, and reports, while Staff assists in booking management.

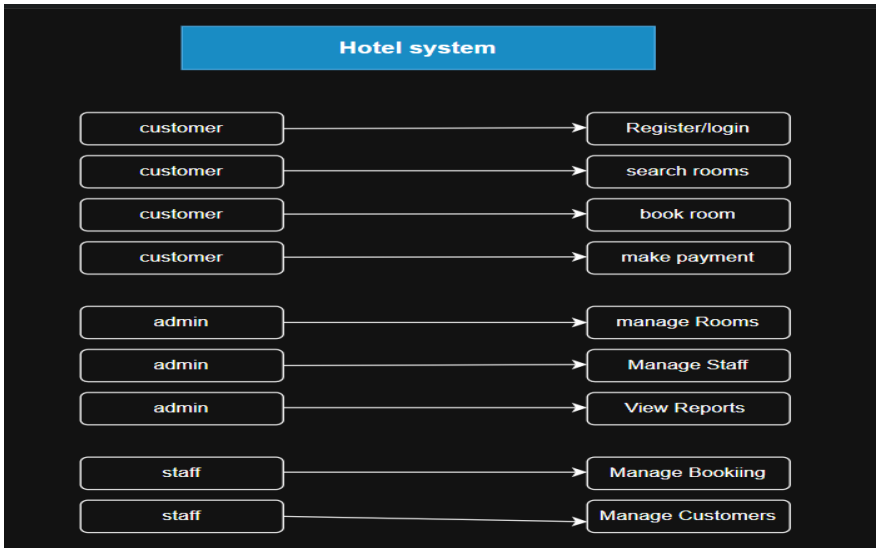


Figure 2.1 Use Case Model Diagram

2.8.1.2 Use Case Documentation

Use Case 1: User Registration

Actor: Customer

Description: The user creates a new account

Pre-condition: User does not have an account

Post-condition: User account is created successfully

Use Case 2: User Login (Security Login)

Actor: Customer / Admin / Staff

Description: The user logs into the system securely

Pre-condition: User must have a registered account

Post-condition: The user gains access to the system

Use Case 3: Book Room

Actor: Customer

Description: The user books an available room

Pre-condition: User must be logged in

Post-condition: The booking is stored in the system

Use Case 4: Make Payment

Actor: Customer

Description: The user completes payment for a booking

Pre-condition: Booking must exist

Post-condition: Payment is recorded

2.8.1.3 Sequence Diagram

The sequence diagram shows how different components interact over time during the booking process.

Narrative Description:

The user sends a request to log in, which is verified by the system. After login, the user searches for rooms, and the system retrieves available room data from the database. The user selects a room and submits a booking request. The system processes the request, stores the booking, and confirms it. Finally, the user completes payment, and the system updates the payment status.

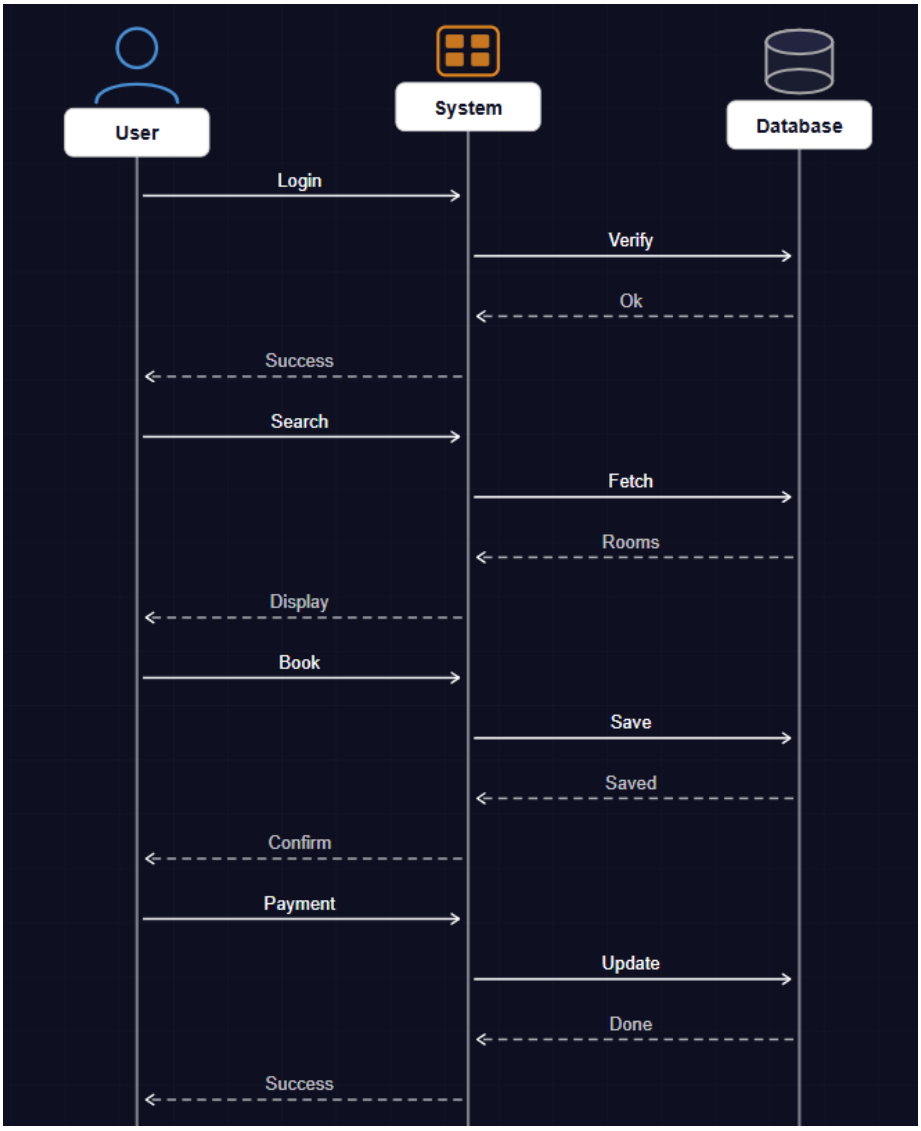


Figure 2.2 Sequence Diagram for Booking Process

2.8.1.4 State Chart Diagram

The state chart diagram represents different states of the booking process and how the system transitions between them.

Narrative Description:

A booking transitions through several states such as “Pending,” “Confirmed,” and “Completed.” Initially, when a booking is created, it is in a pending state. After payment is made, the booking is confirmed. Once the service is completed, the booking status changes to completed.

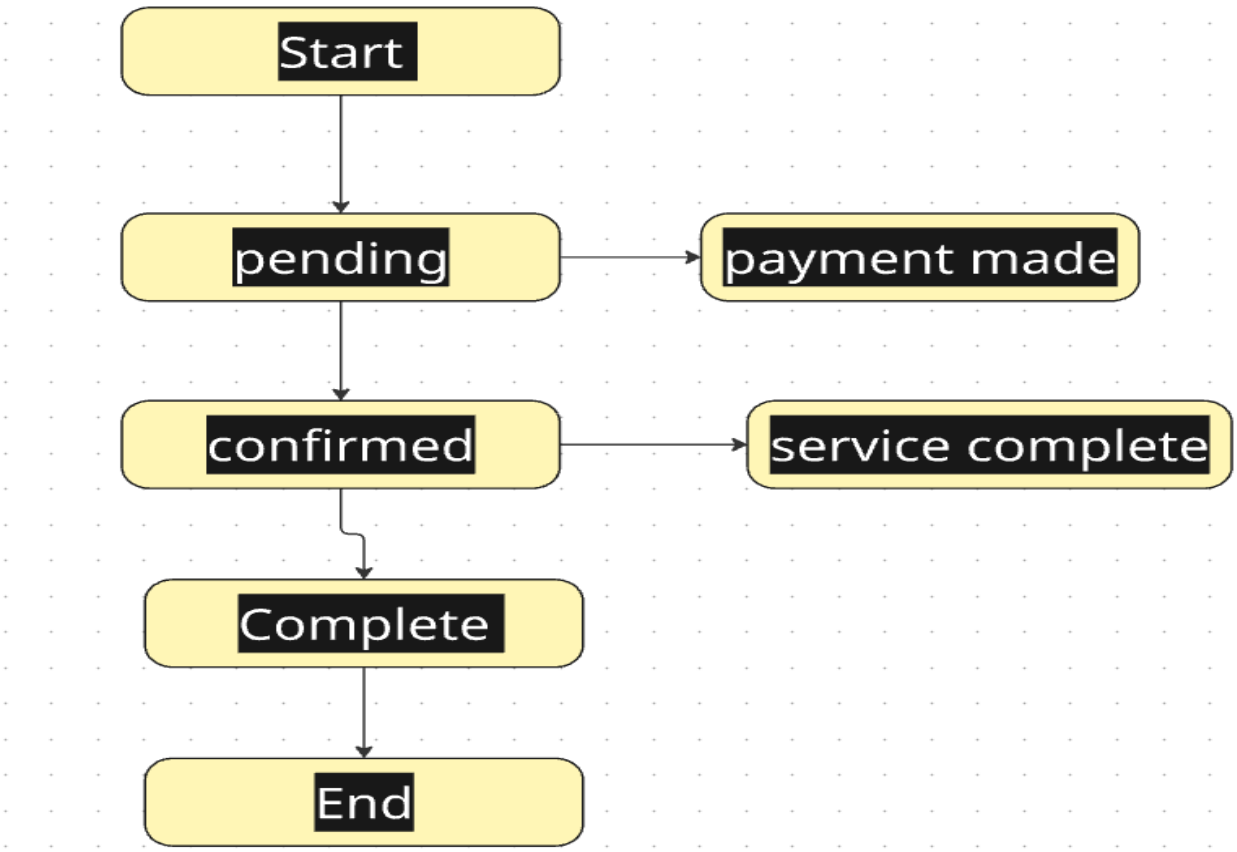


Figure 2.3 State Chart Diagram of Booking Process

2.8.1.5 Activity Diagram

The activity diagram shows the workflow of processes within the system.

Narrative Description:

The process begins when the user logs into the system. The user searches for available rooms and selects one. The system checks availability and allows the user to proceed with booking. After entering details, the user confirms the booking and completes payment. The system then records the transaction and terminates the process.

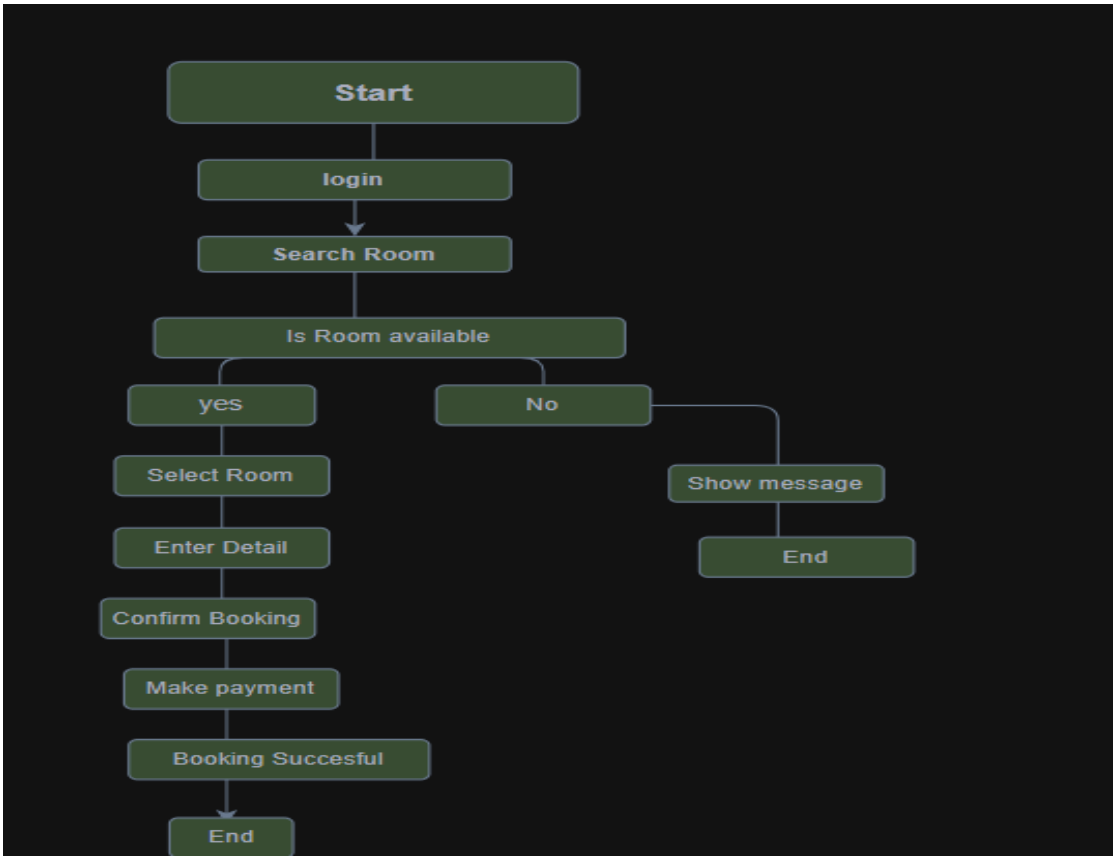


Figure 2.4 Activity Diagram of Booking Workflow

2.8.1.6 Class Diagram

The class diagram represents the structure of the system, including classes, attributes, and relationships.

Narrative Description:

The main classes in the system include Customer, Room, Booking, and Payment. Each class contains attributes such as customer details, room information, booking status, and payment details. Relationships between these classes define how data is managed within the system.

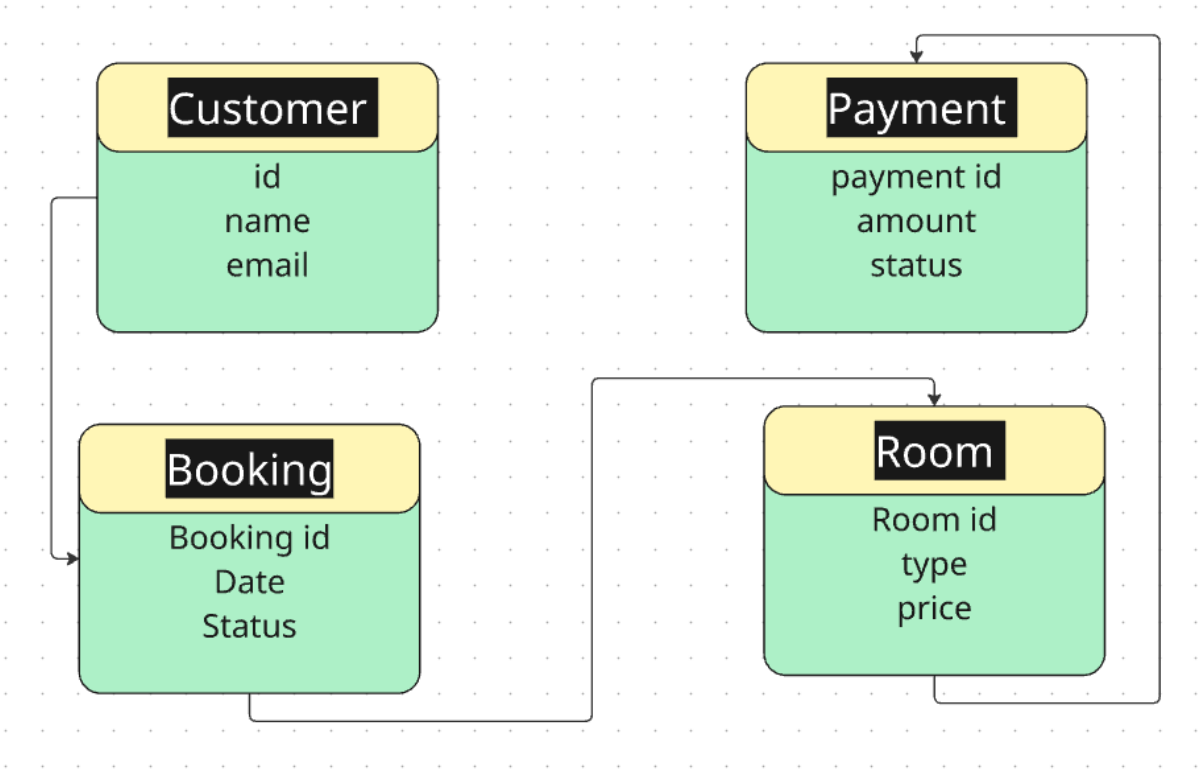


Figure 2.5 Class Diagram of the System

2.8.1.7 User Interface Prototyping

User interface prototyping provides a visual representation of how the system will look and how users will interact with it.

Narrative Description:

The system interface includes several screens such as login page, registration page, dashboard, room listing page, booking form, and payment page. These interfaces are designed to be simple, user-friendly, and easy-to-navigate.

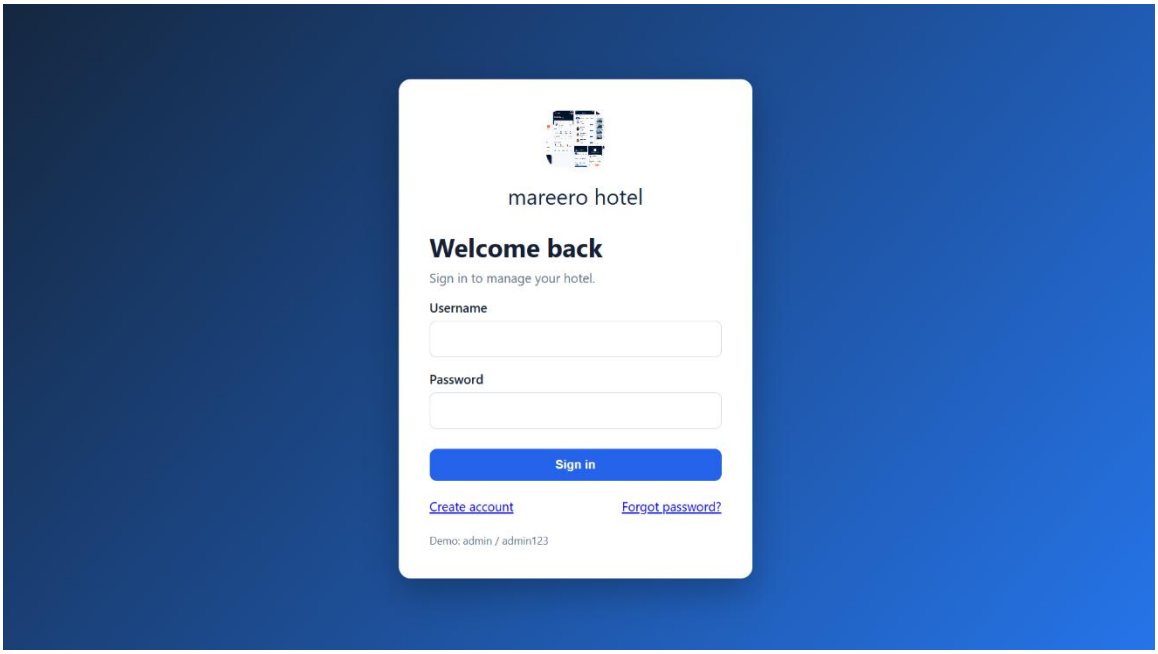


Figure 2.6 Login Page

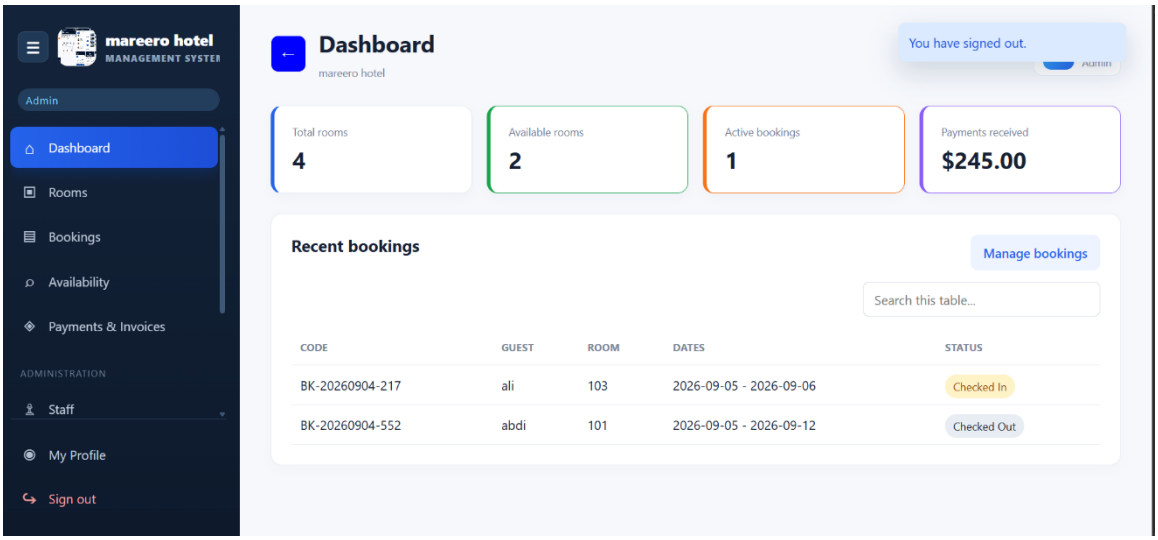


Figure 2.7 Hotel Dashboard

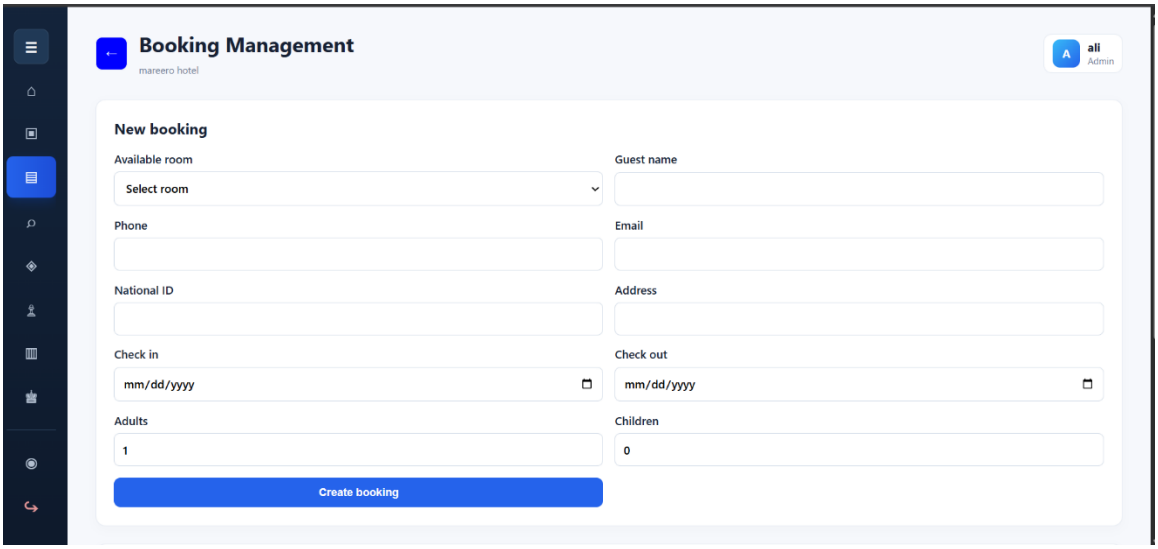


Figure 2.8 Booking Dashboard

2.9 Summary

This chapter has presented an analysis of the existing system and identified its major limitations. It also introduced the proposed system as a solution and defined both functional and non-functional requirements. Additionally, it provided an overview of system analysis to support the system design in the next chapter.

CHAPTER THREE
SYSTEM DESIGN

3.1 Introduction

This chapter presents the system design of the Hotel Booking System. System design is a crucial phase in software development that defines the architecture, components, modules, interfaces, and data flow. The main purpose of this chapter is to transform the system requirements into a structured solution that can be implemented efficiently.

The Hotel Booking System is designed to simplify the process of booking hotel rooms, manage reservations, and improve communication between customers and hotel administrators. This system ensures efficiency, accuracy, and a user-friendly interaction for both users and administrators.

3.2 System Architecture Design

The system follows a three-tier architecture consisting of the following layers:

3.2.1 Presentation Layer (User Interface)

This layer handles user interaction through the web interface and allows users to view rooms, book rooms, and manage their accounts.

- User login page
- Registration form
- Room booking interface
- Admin dashboard

The interface is designed to be simple, responsive, and user-friendly.

3.2.2 Application Layer (Business Logic)

This layer processes all core functionalities such as booking validation, payment processing, and room availability checking.

- User authentication
- Room availability checking
- Booking processing
- Payment handling logic
- Data validation

3.2.3 Data Layer (Database)

This layer is responsible for storing and retrieving data in the database, including user details, room information, bookings, and payments.

- User information
- Room details
- Booking records
- Payment records

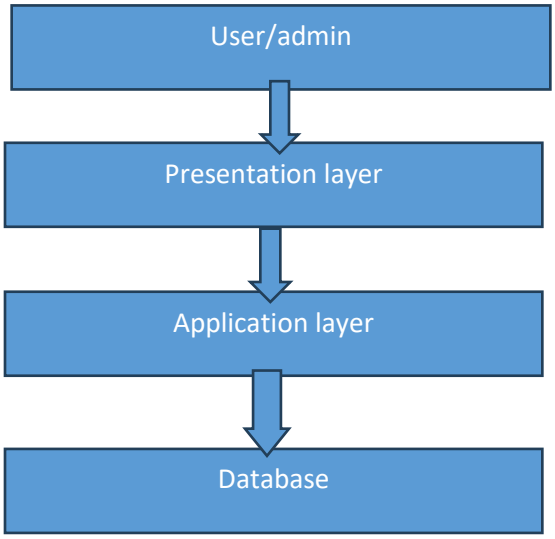
- Admin data

The database ensures data consistency and security.

3.2.4 Architecture Diagram

The architecture diagram illustrates the three-tier structure of the system, including presentation layer, business logic layer, and database layer.

The presentation layer handles user interaction, the application layer processes business logic, and the data layer manages data storage and retrieval**.**



3.3 System Modules Design

The system is divided into different modules to improve maintainability and scalability.

3.3.1 User Module

This module allows users to:

- Register an account
- Log in to the system
- Search available rooms
- Book rooms
- Cancel a booking
- View booking history

3.3.2 Admin Module

This module is used by hotel administrators to:

- Manage rooms (add, update, delete)

- Manage bookings
- Manage users
- View reports
- Confirm or reject bookings

3.3.3 Booking Module

This module handles:

- Room reservation process
- Availability checking
- Booking confirmation
- Booking cancellation

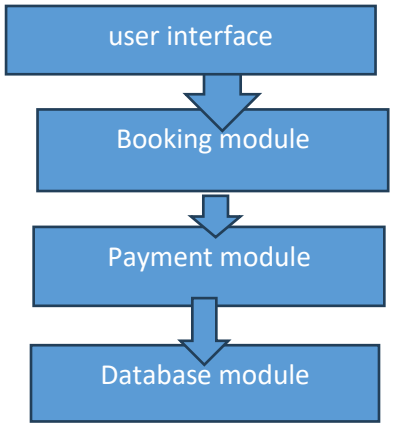
3.3.4 Payment Module

This module manages:

- Payment processing
- Payment confirmation
- Receipt generation

3.3.5 Component Diagram

The component diagram illustrates how different modules such as user management, booking, payment, and reports interact through defined interfaces within the system.



3.4 System Design Methodology

The system was designed using the Object-Oriented Design (OOD) approach. This methodology helps organize the system into classes and objects that interact with each other.

Key principles of OOD used in the system include:

- Encapsulation

- Modularity
- Reusability
- Maintainability
- Inheritance
- Abstraction

Examples of classes used in the system include:

- User
- Admin
- Room
- Booking
- Payment

3.5 Data Flow Design

The system data flow describes how information flows within the system.

3.5.1 Level 0 Data Flow Diagram (Context Diagram)

The system interacts with two main external entities:

- Customer
- Admin

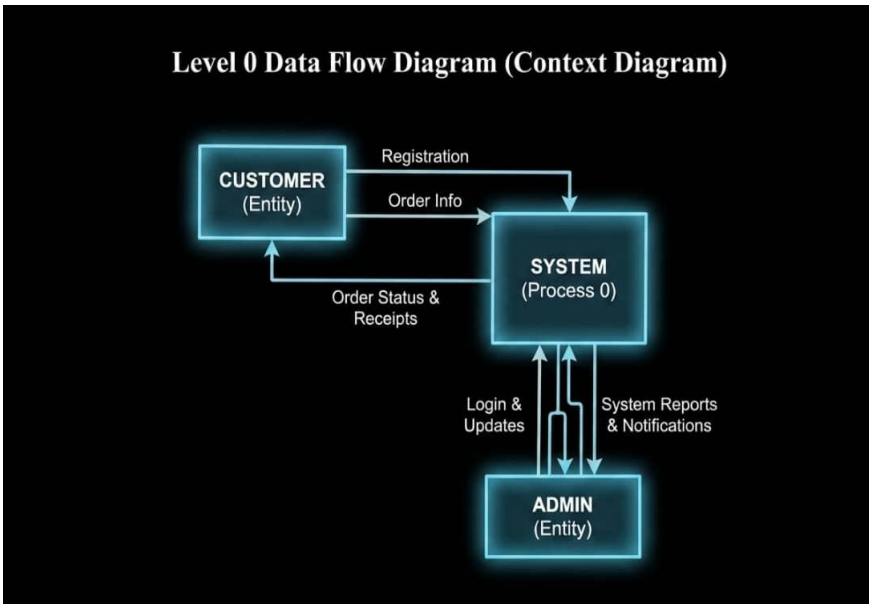


Figure 3.1 Level 0 Data Flow Diagram

Customers send booking requests, and the system responds with confirmations. The admin manages system data and receives reports.

3.5.2 Level 1 Data Flow Diagram

This includes:

- User authentication process
- Booking process
- Room management process
- Payment processing

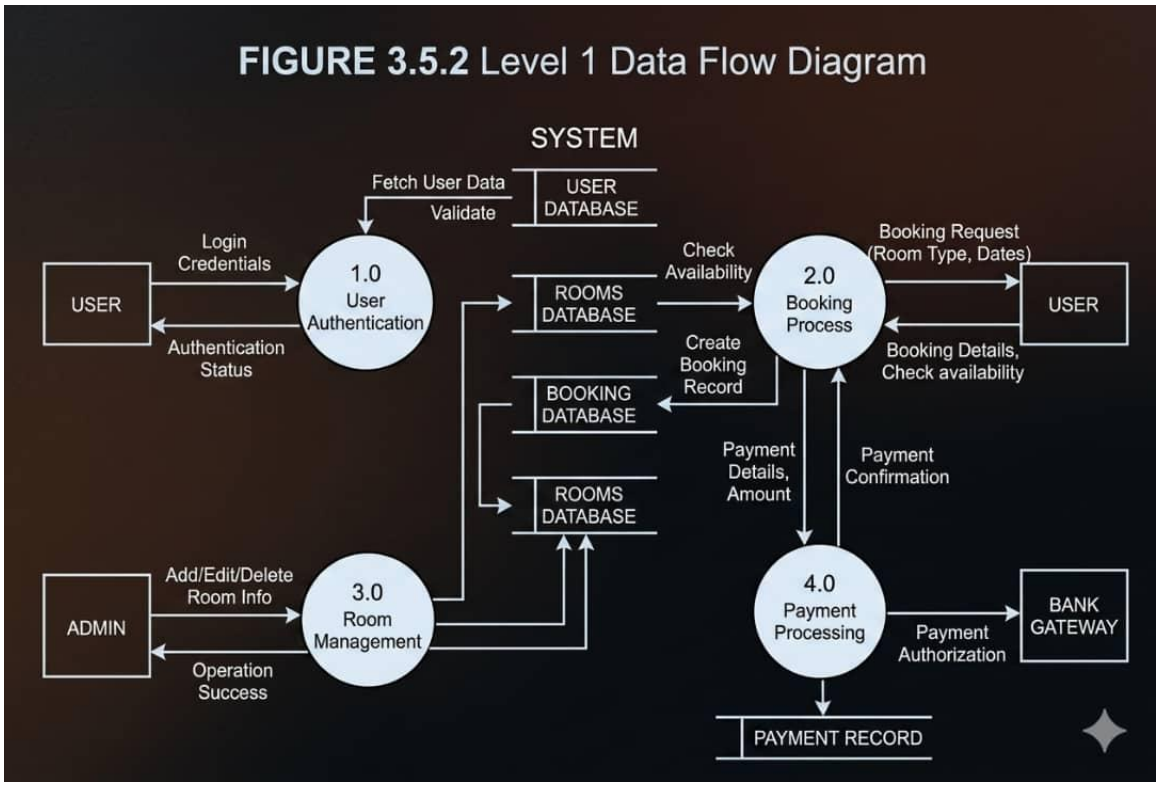


Figure 3.2 Level 1 Data Flow Diagram

3.6 Database Design

The system database is designed using relational database principles.

3.6.1 Main Tables

User Table

Field Name	Data Type	Key
UserID	**INT**	**PK**
FullName	**VARCHAR**	
Email	**VARCHAR**	
Phone	**VARCHAR**	
Password	**VARCHAR**	
Role	**VARCHAR**	
CreatedAt	**DATETIME**	

Table 3.1 Users Table

Table 3.1 Rooms Table

Rooms Table

Field Name	Data Type	Key
RoomID	**INT**	**PK**
RoomNumber	**VARCHAR**	
RoomType	**VARCHAR**	
Price	**DECIMAL**	
Status	**VARCHAR**	
Capacity	**INT**	

Table 3.2 Booking Table

Booking Table

Field Name	Data Type	Key
BookingID	**INT**	**PK**
UserID	**INT**	**FK**
RoomID	**INT**	**FK**
CheckInDate	**DATE**	
CheckOutDate	**DATE**	
TotalAmount	**DECIMAL**	
Status	**VARCHAR**	

Table 3.3 Payments Table

Payments Table

Field Name	Data Type	Key
PaymentID	**INT**	**PK**
BookingID	**INT**	**FK**
Amount	**DECIMAL**	
PaymentDate	**DATETIME**	
Method	**VARCHAR**	
Status	**VARCHAR**	

Table 3.4 Admin Table

Admin Table

Field Name	Data Type	Key
AdminID	**INT**	**PK**
Name	**VARCHAR**	
Email	**VARCHAR**	
Password	**VARCHAR**	

3.6.2 Entity Relationship Diagram

An Entity Relationship Diagram (ERD) is a graphical representation used to model the structure of a database system. It illustrates system entities, their attributes, and the relationships among them. ERDs are important during the database planning stage because they help developers understand how data is organized and connected before implementation.

In this system, entities such as User, Room, Booking, and Payment are connected through relationships that define how booking and payment processes are managed.

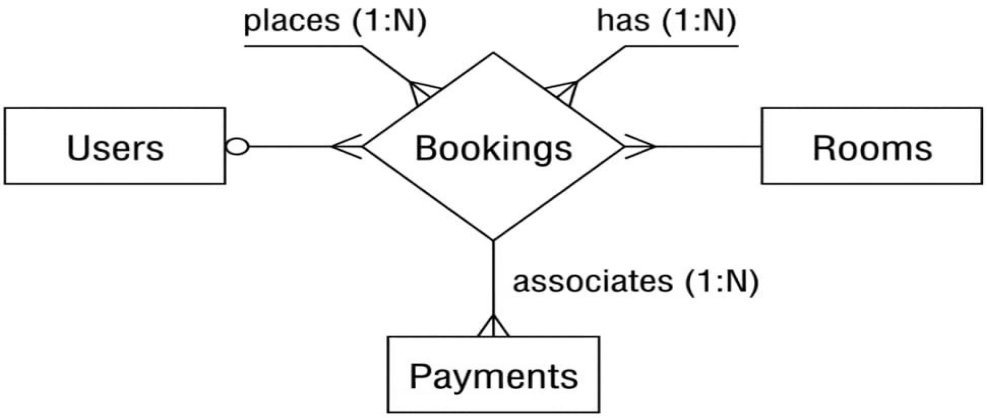


Figure 3.3 Entity Relationship Diagram

3.7 System Interface Design

The interface design focuses on usability and simplicity.

3.7.1 User Interface

- A home page displaying available rooms
- Search bar for rooms
- Booking form
- Login and registration pages

3.7.2 Admin Interface

- Dashboard showing statistics
- Room management panel
- Booking management section
- Reports section

3.8 Security Design

Security is an important part of the system. The following security measures are implemented:

- Password hashing using BCrypt
- User authentication and authorization

- Role-based access control
- Secure access to the database
- Session management
- Input validation to prevent SQL injection attacks
- HTTPS secure connection

3.9 System Requirements

3.9.1 Hardware Requirements

- Processor: Intel Core i3 or higher recommended
- RAM: 4GB minimum
- Storage: 500GB HDD or higher

3.9.2 Software Requirements

Operating System: Windows or Linux

Database: MySQL

- Programming Language: PHP
- Web Browsers: Google Chrome and Mozilla Firefox

3.9.3 Deployment Diagram

The deployment diagram explains how the system is deployed on a server, client devices, and a connected database.

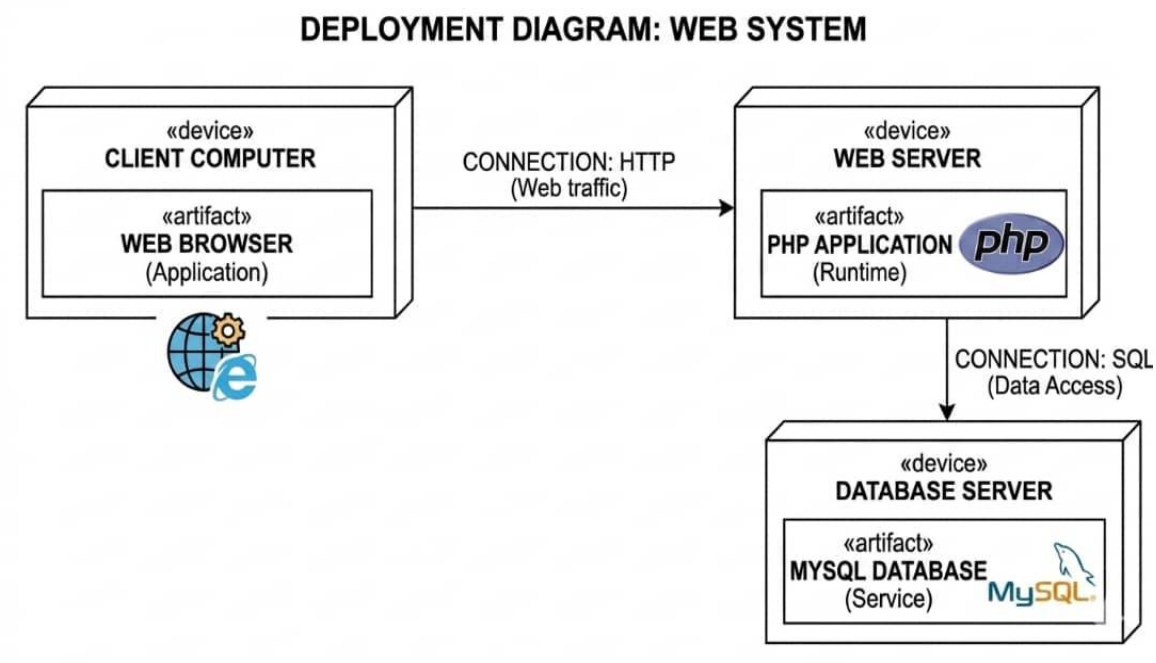


Figure 3.4 Deployment Diagram

3.10 Summary

This chapter has presented the overall system design of the Hotel Booking System. It explained the system architecture, modules, database design, interface design, and security mechanisms. The design ensures that the system is efficient, scalable, secure, and easy to use. The next chapter will focus on system implementation and testing.

CHAPTER FOUR
IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter presents the implementation and testing of the Hotel Booking Management System developed from the requirements and design described in the previous chapters. The system was implemented as a web application using PHP for server-side processing, MySQL for relational data storage, HTML for page structure, CSS for interface styling, and JavaScript for interactive functions. The chapter explains the development environment, database implementation, major modules, testing activities, user guidance, and installation procedure.

The implementation focused on converting the approved system design into a functional application that supports room management, booking management, room availability searches, customer payments, invoice generation, staff records, reports, user authentication, user profiles, and hotel branding. Testing was performed at component, integration, and complete-system levels to confirm that data moves correctly between the user interface, application logic, and database.

4.2 System Implementation

System implementation involved creating the database, establishing a secure connection between PHP and MySQL, developing reusable layout components, and implementing each functional module. A modular file structure was used so that configuration, navigation, styling, data operations, and individual application pages remain easy to understand and maintain.

4.2.1 Development Environment

The system was developed and tested on a Windows environment using XAMPP services. Apache provides the local web server, while MySQL stores the application data. PHP 8.2 executes the server-side logic, and phpMyAdmin can be used to import and inspect the database. The interface is accessed through a modern web browser. Visual Studio Code or another text editor may be used to maintain the source files.

- Programming language: PHP 8.2

- Database management system: MySQL
- Web server package: XAMPP with Apache and MySQL
- Client technologies: HTML5, CSS3, and JavaScript
- Database administration: phpMyAdmin
- Supported browsers: Google Chrome, Microsoft Edge, and Mozilla Firefox

4.2.2 Database Implementation

The database was implemented under the name hotel_management. It contains related tables for users, room types, rooms, customers, bookings, payments, staff, password-reset requests, and hotel settings. Primary keys uniquely identify records, while foreign keys connect rooms to room types, bookings to customers and rooms, and payments to bookings. Referential constraints reduce inconsistent records and preserve the relationship between transactions.

Prepared PDO statements are used when values originate from forms. This approach separates SQL instructions from user input and reduces the risk of SQL injection. Passwords are stored as secure hashes rather than readable text. The database script also creates an initial administrator account and sample room types to simplify first-time setup.

4.2.3 Authentication and Access Control

Authentication begins on the login page. The system retrieves the account matching the submitted username and verifies the password against its stored hash. After successful authentication, the user's identifier, name, and role are stored in a server-side session. Protected pages call a shared login check before displaying content. Administrative pages apply an additional role check so that receptionists cannot access sensitive configuration and reporting functions.

The registration page creates receptionist accounts, while the profile page allows a signed-in user to update a name, username, email address, or password. The forgot-password page records a time-limited reset request that can be connected to an email delivery service during deployment.

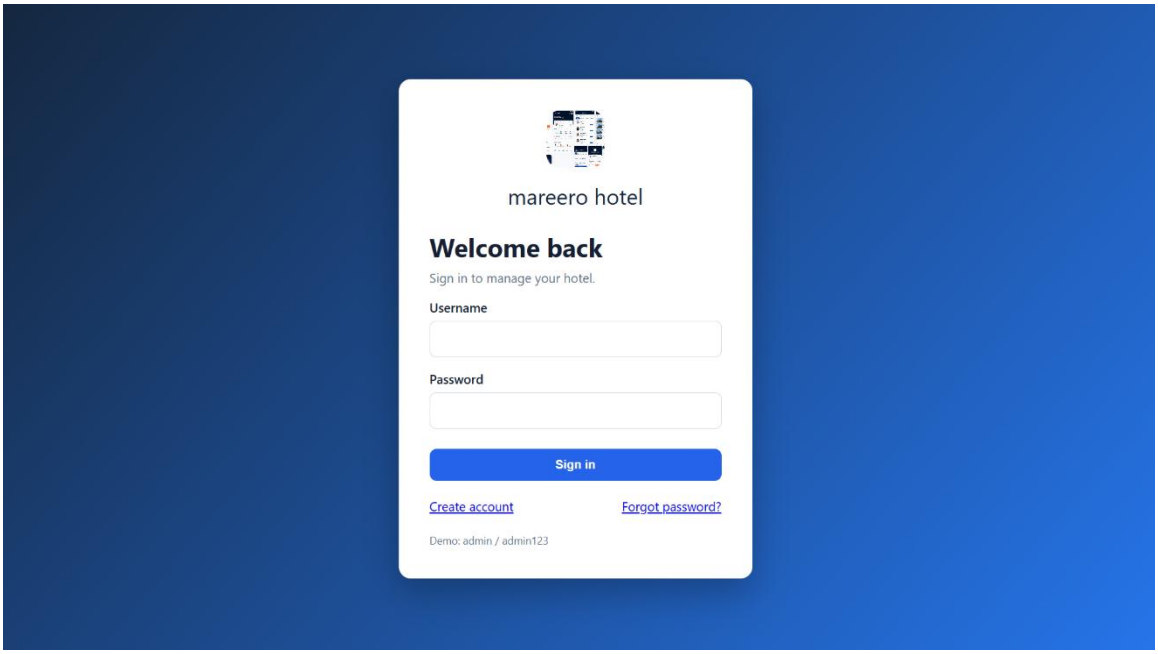


Figure 4.1 Login Page

4.2.4 Dashboard Implementation

The dashboard provides a summary of hotel operations. It displays the total number of rooms, available rooms, active bookings, and received payments. It also presents recent bookings and a revenue overview. These values are calculated directly from the database, allowing the page to reflect current operational data whenever it is loaded.

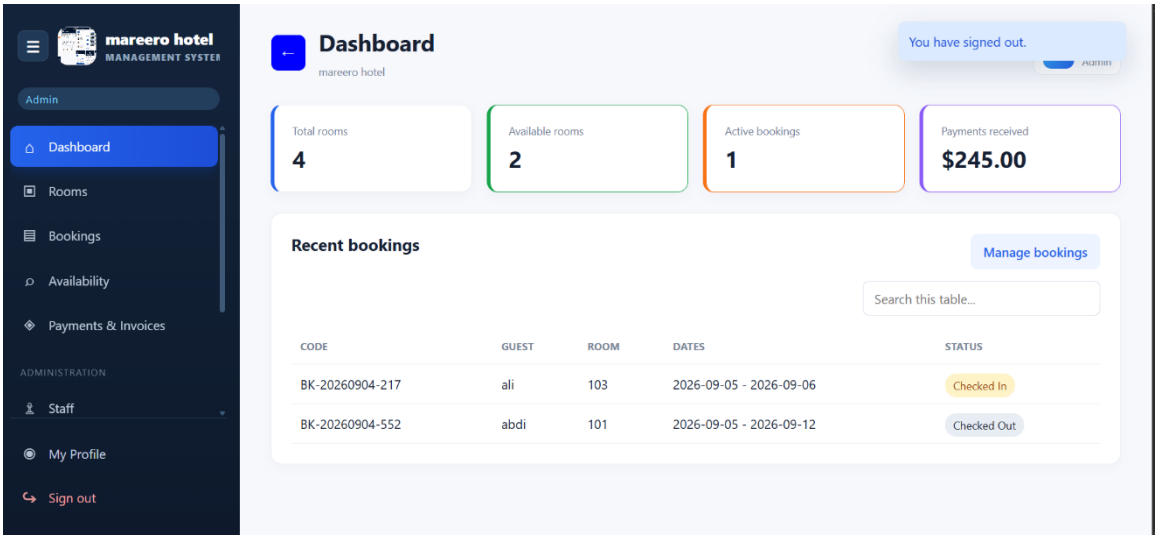


Figure 4.2 System Dashboard

4.2.5 Room Management Implementation

The room management module enables an administrator to add rooms, update room details, and delete records that are no longer required. Each room contains a unique room number, room type, nightly price, operational status, and optional notes. Room status values include Available, Booked, and Under Maintenance. Room types provide reusable classifications such as Single, Double, and Deluxe.

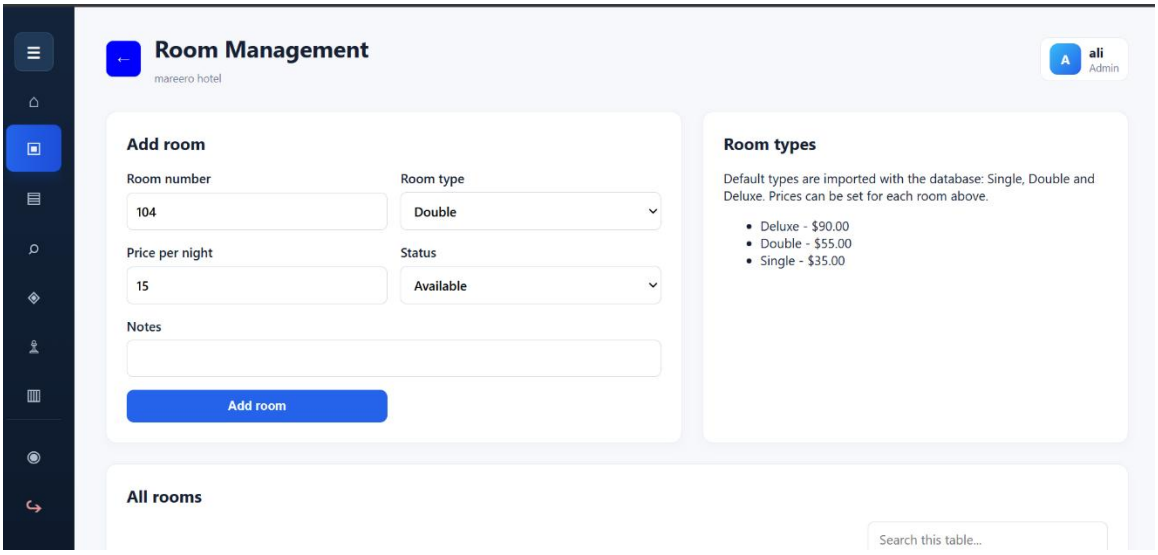


Figure 4.3 Room Management Page

4.2.6 Booking and Availability Implementation

The booking module captures guest information, selected room, check-in date, check-out date, number of adults, and number of children. The system calculates the number of nights and multiplies it by the room price to obtain the booking total. A unique booking code is generated for identification. Staff may update an active booking to Checked In or Checked Out, or cancel it when necessary.

The availability search compares the requested dates against active bookings. A room is excluded when an existing booking overlaps the requested date range. Rooms under maintenance are also excluded. The search may be filtered by room type, ensuring that users view only suitable rooms for the requested period.

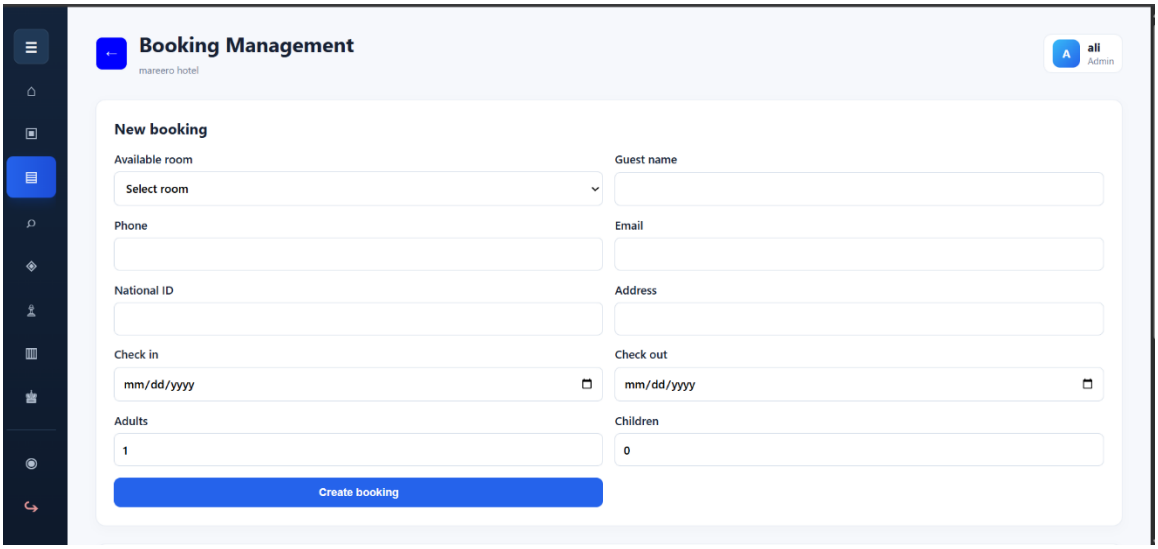


Figure 4.4 Booking Management Page

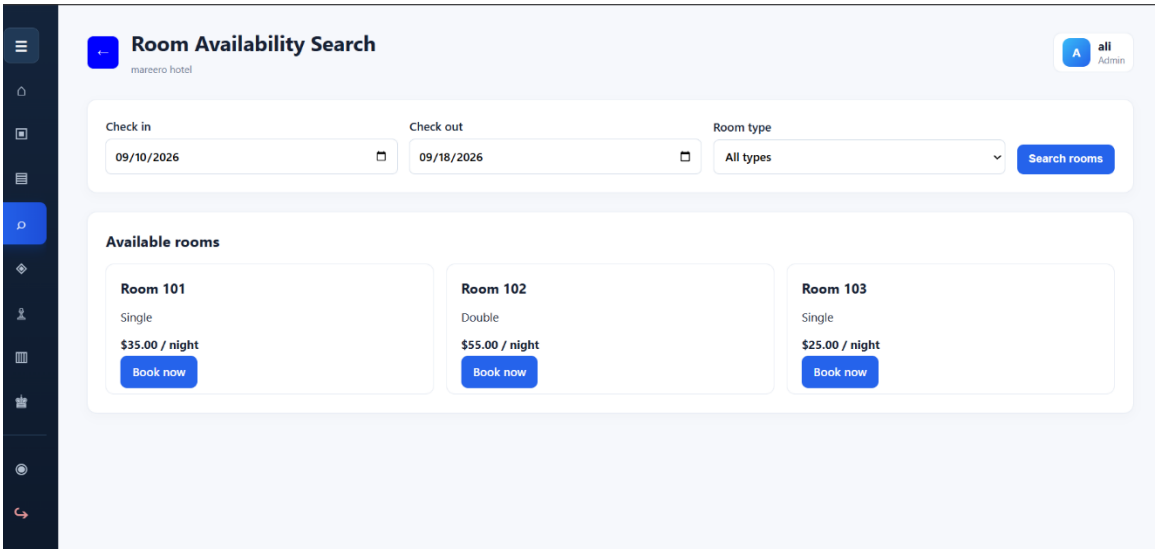


Figure 4.5 Room Availability Search

4.2.7 Payment and Invoice Implementation

The payment module records the booking reference, amount, payment method, payment status, payment date, and optional notes. Supported methods include cash, card, mobile money, and bank transfer. Payment status can be Pending, Paid, or Completed. The payment history table enables authorized users to review recorded transactions, while the invoice page displays customer, room, booking, and payment information in a printable

format.

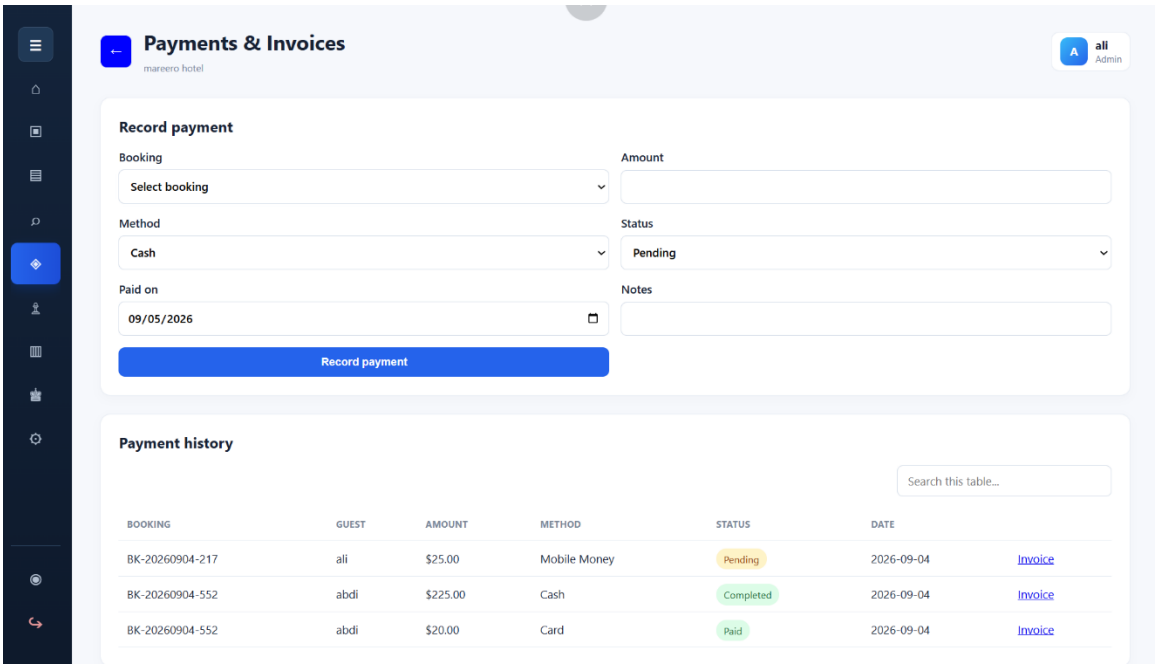


Figure 4.6 Payment and Invoice Page

4.2.8 Staff, Reports, and Hotel Settings

The staff module stores employee names, contact information, roles, responsibilities, and employment status. The reports module summarizes bookings, cancellations, received revenue, and revenue grouped by room. Hotel settings allow the administrator to change the hotel name, email address, phone number, location, and logo. Because branding is stored in the database rather than fixed in the source code, the same application can be

configured for different hotels.

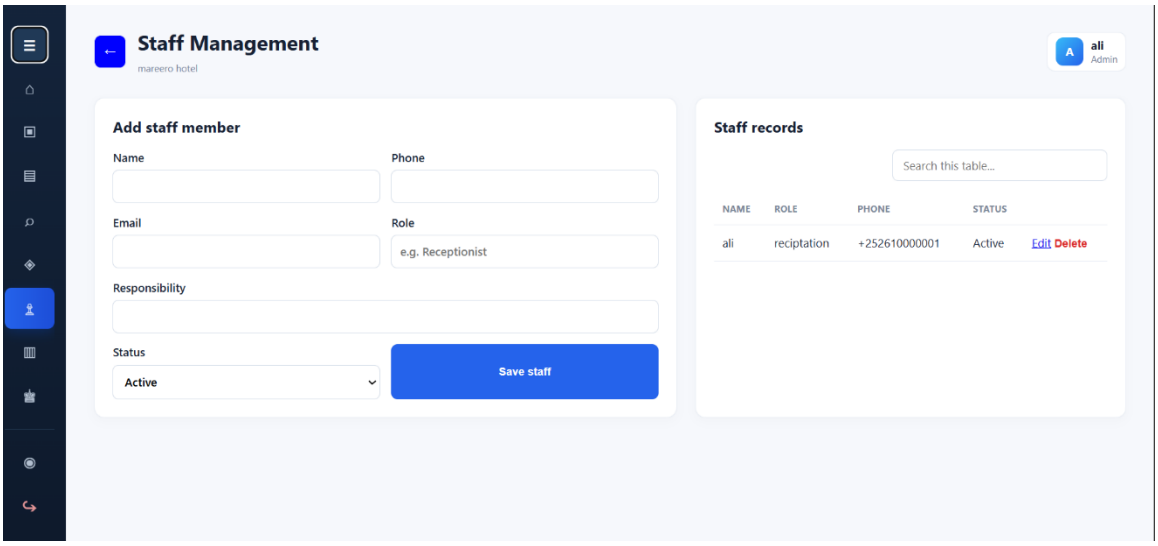


Figure 4.7 Staff Management Page

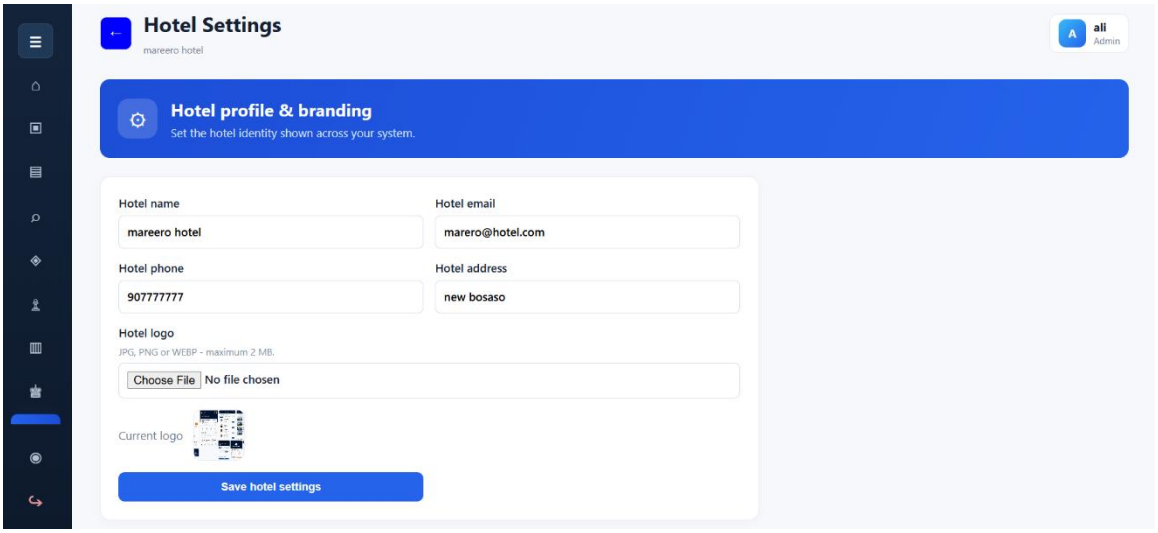


Figure 4.8 Hotel Settings Page

4.3 Application Testing

Application testing was conducted to determine whether the implemented modules meet the functional requirements. Each test used a defined input or action, an expected result, and an observed result. The principal test cases are summarized below.

No.	Test Case	Expected Result	Actual Result	Status
1	Valid administrator login	Dashboard opens with admin menu	Dashboard and admin menu displayed	Pass
2	Invalid login details	Access is rejected with an error	Error message displayed	Pass
3	Register receptionist	New account is stored	Account created successfully	Pass
4	Add a room	Room appears in room list	Room saved and displayed	Pass
5	Update room details	New values replace old values	Room values updated	Pass
6	Search available room by dates	Only non-overlapping rooms appear	Available rooms displayed	Pass
7	Create booking	Booking and customer are stored	Booking code generated	Pass
8	Cancel booking	Booking is cancelled and room released	Status and room updated	Pass
9	Record payment	Payment appears in history	Payment stored successfully	Pass
10	Generate invoice	Correct invoice details appear	Printable invoice displayed	Pass
11	Receptionist opens admin page	Access is denied	User returned to dashboard	Pass
12	Change hotel name and logo	Branding updates across pages	Updated branding displayed	Pass

Table 4.1 System Test Results

4.3.1 Unit Testing

Unit testing examined individual operations such as password verification, room insertion, booking-total calculation, status changes, payment recording, and hotel-setting updates. PHP syntax validation was also applied to the application files. The tested operations produced the expected database changes and page responses.

4.3.2 Integration Testing

Integration testing verified the interaction between related modules. Creating a booking connects a customer to a room, while recording a payment connects the transaction to the selected booking. Cancelling or checking out a booking releases the room for future use. Dashboard and report values were checked after transactions to confirm that they reflect the same database records.

4.3.3 System Testing

System testing examined the complete workflow from login through room search, booking, payment, invoice generation, reporting, and logout. Navigation, responsive layout, session control, validation messages, and role restrictions were reviewed as part of the test. The system completed the required workflows without critical errors in the local test environment.

4.3.4 User Acceptance Testing

User acceptance testing focuses on whether hotel staff can complete routine tasks with limited technical knowledge. The interface uses consistent menus, descriptive labels, status indicators, confirmation messages, and searchable tables. Final acceptance should be completed with representative hotel users after deployment, and their comments should be recorded for future improvements.

4.4 Hardware and Software Acquisition

The system uses commonly available hardware and open-source software. A computer with at least an Intel Core i3 processor, 4 GB of memory, and adequate storage can support local operation for a small hotel. The required software includes a Windows or Linux operating system, Apache, PHP, MySQL, and a web browser. XAMPP provides the main server components in one installation package, which reduces setup cost and complexity.

4.5 User Manual Preparation

The user manual describes the actions required to operate the application. Users should first start the Apache and MySQL services and open the system address in a web browser. After login, the sidebar provides access to the modules permitted by the user's role.

- Login: enter a registered username and password, then select Sign In.
- Rooms: add a room or select Edit to update its number, type, price, status, or notes.
- Availability: choose check-in and check-out dates, optionally select a room type, and run the search.
- Bookings: select an available room, enter guest and stay details, and create the booking.

- Payments: select a booking, enter payment details, save the payment, and open its invoice if required.
- Staff and reports: administrators may manage employees and review booking or revenue summaries.
- Hotel settings: administrators may update the hotel name, contact details, address, and logo.
- Profile and logout: every signed-in user may update personal account information and securely end the session.

4.6 Installation Process

- Install XAMPP and start the Apache and MySQL services.
- Copy the project app folder into the XAMPP htdocs directory.
- Open phpMyAdmin and import database/hotel_management.sql.
- Confirm the database values in includes/config.php. The default local configuration uses host localhost, database hotel_management, username root, and an empty password.
- Open the project URL in a browser, for example http://localhost/project%20app/.
- Sign in with the initial administrator account and immediately change the account details and hotel settings.
- Test room, booking, payment, staff, and reporting operations before using the system with live hotel records.

4.7 Summary

This chapter described the implementation of the Hotel Booking Management System using PHP and MySQL. It presented the development environment, database structure, security controls, core modules, test results, user instructions, and installation process. The implemented application supports the principal requirements identified in Chapter Two and follows the system design presented in Chapter Three.

CHAPTER FIVE

CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusions

The Hotel Booking Management System was developed to replace slow and error-prone manual hotel operations with a centralized digital solution. The completed application provides secure user access and practical modules for rooms, bookings, availability searches, payments, invoices, staff, reports, user profiles, and hotel branding. These functions address the main problems identified in the existing system, including delayed record retrieval, inaccurate room status, duplicate or conflicting reservations, weak reporting, and dependence on paper records.

The project demonstrates that PHP and MySQL can provide an affordable and maintainable platform for a small or medium-sized hotel. The relational database keeps operational records connected, while the role-based interface limits administrative functions to authorized users. Date-based availability checking, booking status management, payment history, printable invoices, and operational summaries improve accuracy and support daily decision-making.

Testing showed that the main workflows operate together as intended in the local development environment. The system can be configured with a hotel's own name, contact information, and logo, allowing the same software package to be adapted for different hotel installations. The general objective of designing and developing an understandable hotel booking management system was therefore achieved.

5.2 Recommendations

Although the current system satisfies the defined core requirements, the following improvements are recommended for future development and deployment:

- Deploy the application on a secure web server and enable HTTPS to protect login and transaction data.
- Integrate an email or SMS service for booking confirmations, password-reset links, reminders, and cancellation notices.
- Add an online payment gateway for verified card and mobile-money transactions.

- Introduce automated database backups and a tested recovery procedure to reduce the risk of data loss.
- Add detailed audit logs that record important changes made by administrators and receptionists.
- Expand reporting with date ranges, occupancy rates, downloadable PDF reports, and spreadsheet export.
- Provide a customer-facing reservation portal while keeping the current administrative interface for hotel staff.
- Conduct usability and security testing with real hotel personnel before full production deployment.
- Review room pricing, tax, discount, cancellation, and refund rules with each hotel before operational use.
- Maintain the PHP and MySQL environment with supported versions and regularly review access permissions.

Future work should be introduced incrementally and tested against the existing booking workflow. Priority should be given to security, backup, customer notifications, and operational reporting because these areas have the greatest effect on safe and dependable hotel service.

REFERENCES

PHP Documentation. (2026). PHP Manual. https://www.php.net/manual/en/

Oracle Corporation. (2026). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

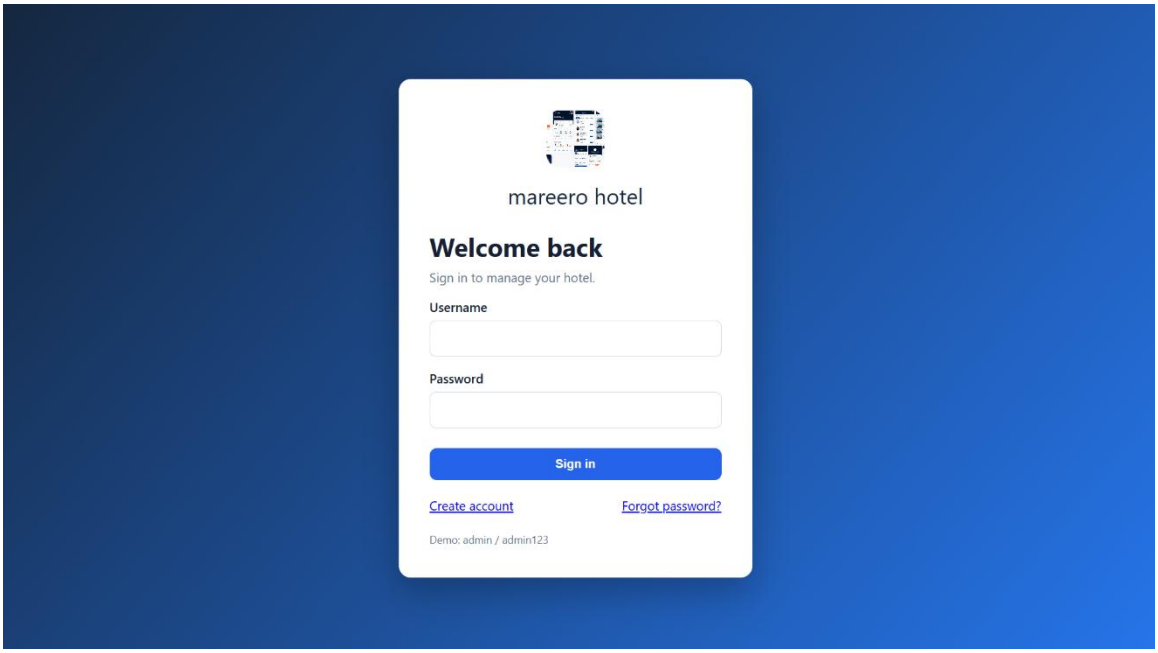
OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

Gollis University. (2026). Final Year Proposal and Project Report Guidelines. Faculty of Computer Studies.

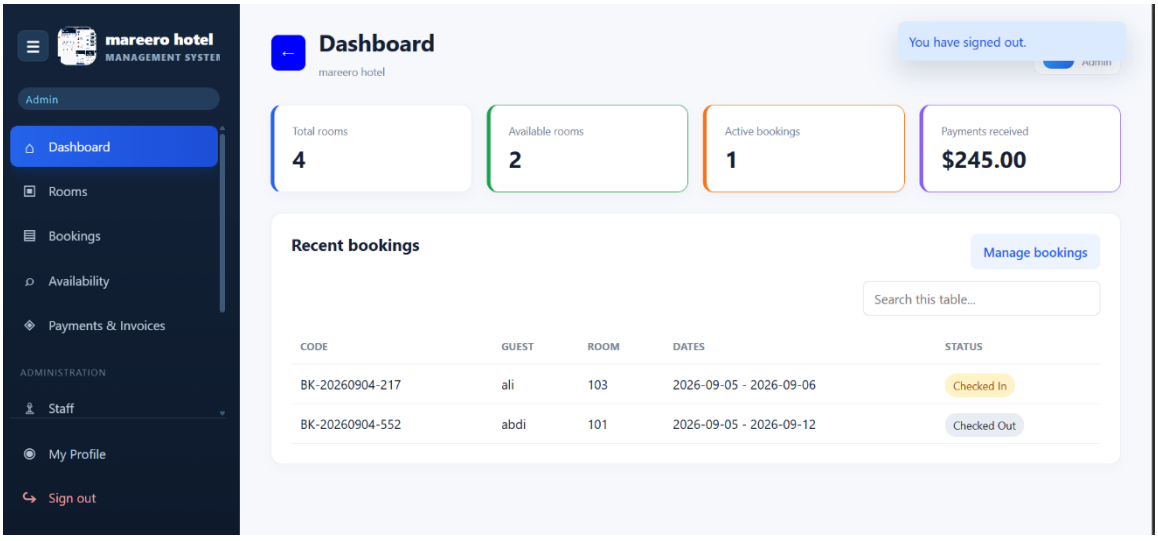
APPENDICES

Appendix A System Screenshots

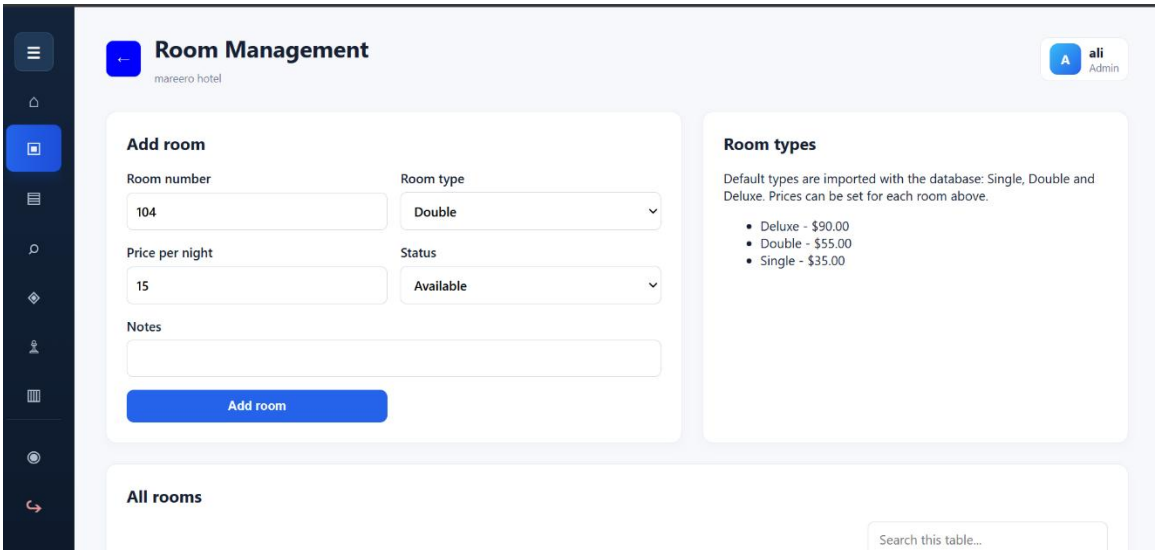
Replace the placeholders below with final screenshots captured from the completed system. Keep each caption with its related screenshot and update the List of Figures in Microsoft Word.



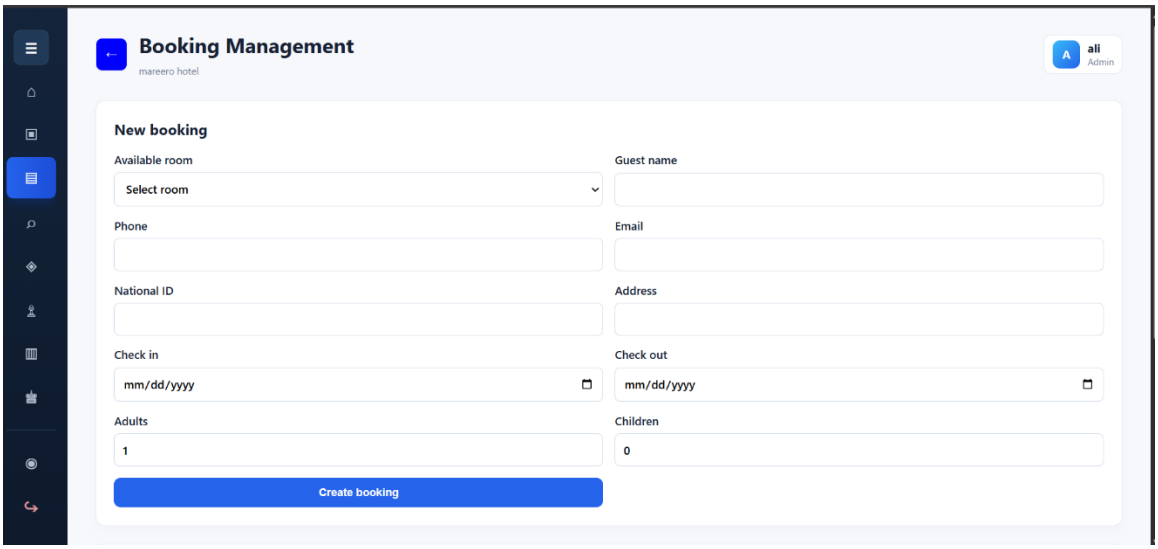
Appendix Figure A.1 Login Page



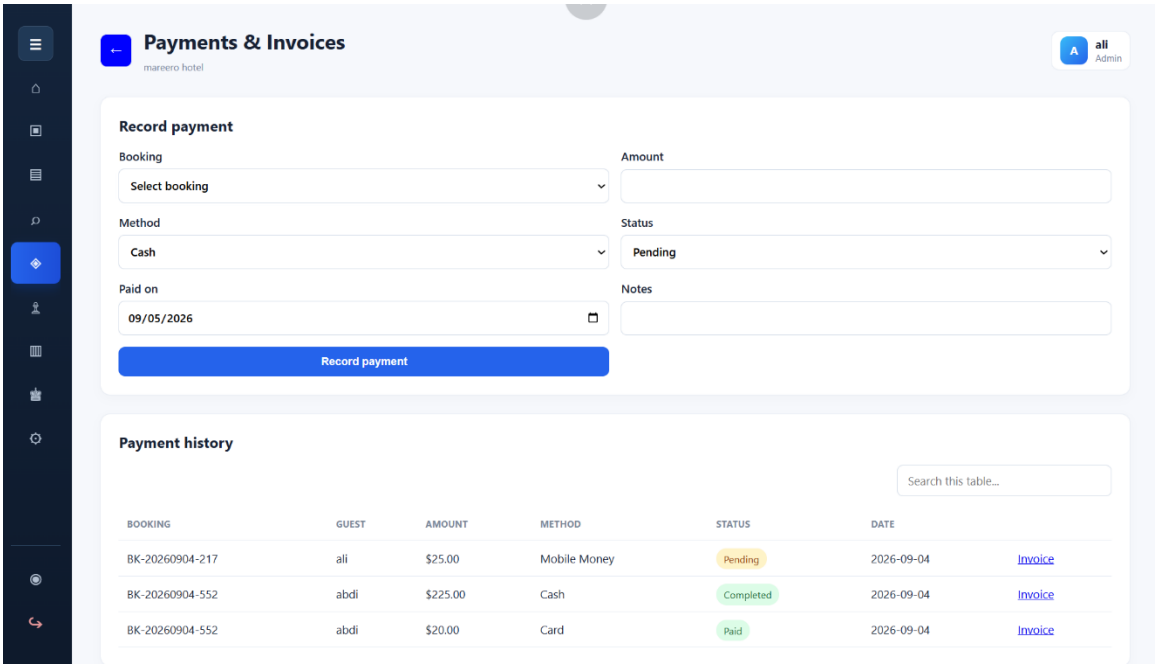
Appendix Figure A.2 Dashboard



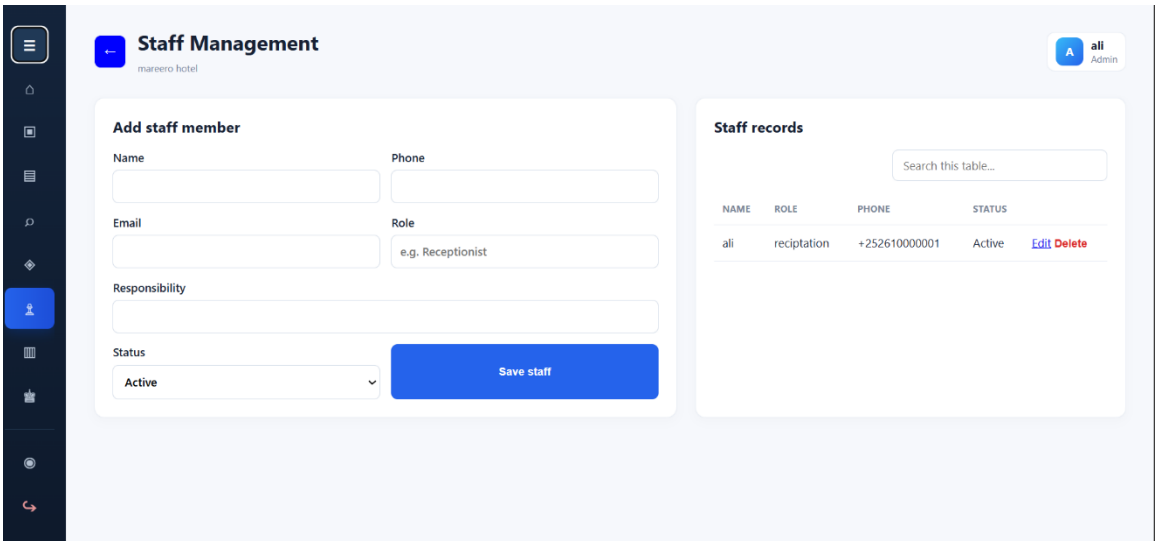
Appendix Figure A.3 Room Management



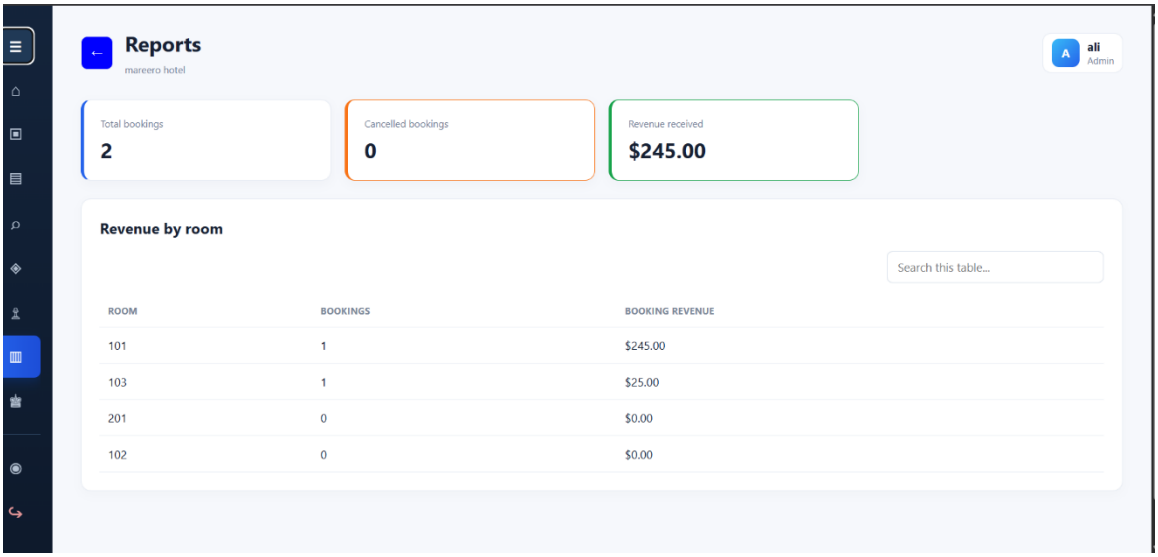
Appendix Figure A.4 Booking Management



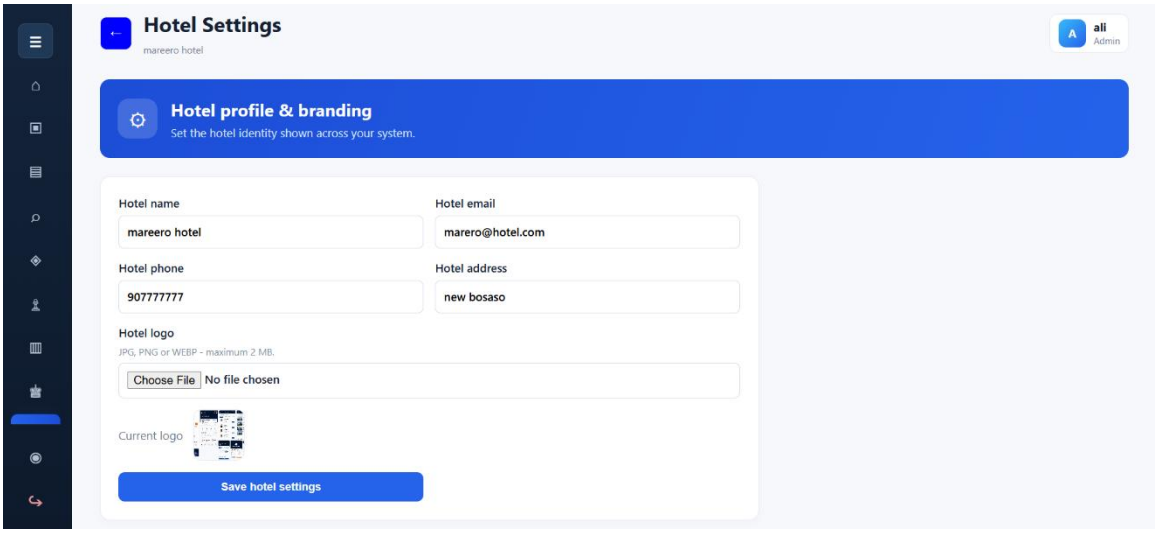
Appendix Figure A.5 Payment and Invoice



Appendix Figure A.6 Staff Management



Appendix Figure A.7 Reports



Appendix Figure A.8 Hotel Settings

Appendix B Source Code and Database

The complete PHP source code, CSS assets, SQL database script, installation instructions, and supporting files are supplied electronically with the project deliverables. Representative code samples may be inserted here if requested by the supervisor.